Guion de la Presentacion: Pocket Cube 2x2x2

Diapo 1: Introducción

Hola a todos, me llamo Adrian y hoy voy a presentar mi proyecto del Pocket Cube. No soy un experto en algoritmos, de hecho es la primera vez que toco estas cosas, asi que he requerido mas apoyo del que me gustaria con GPT6, sobre todo para el codigo.

Diapo 2: Como representamos el cubo

Un cubo 2x2x2 tiene ocho piezas en las esquinas. No hay centros ni aristas, asi que el modelo solo guarda ocho posiciones. Usamos dos cosas llamadas permutacion y orientacion. Permutacion es donde esta cada pieza, y orientacion es como esta girada.

Diapo 3: La regla del modulo 3

Esto es un poco dificil de entender pero sumamente util. La orientacion siempre suma 0 modulo 3. Esto significa que si sabemos como estan giradas siete esquinas, la octava se calcula sola. Asi el programa tiene menos cosas que calcular y va mas rapido, asi como requiere menos memoria haciendolo mas eficiente.

Diapo 4: Moviendo las piezas

Los movimientos son las letras de siempre: U, R, F, D, L y B. Para no tener que programar cada movimiento a mano, lo que hace el codigo es usar unas tablas. Estas tablas te dicen de donde viene la esquina y cuanto se gira al moverse. Hacemos unas pruebas y si funciona continuamos..

Diapo 5: Por que no usar BFS o DFS

Al principio considere alternativas como BFS o DFS. BFS enuentra la mejor solucion pero gasta mucha memoria porque guarda demasiadas cosas. DFS gasta poca memoria pero se pierde por caminos larguisimos. Y el famoso A* necesita guardar demasiados estados tambien, asi que descarte esos metodos.

Diapo 6: La magia de IDA*

Aqui entra IDA*, que es el algoritmo que usamos. Es una busqueda en profundidad pero le vamos subiendo el limite poco a poco. Lo bueno es que gasta poca memoria y nos asegura encontrar la solucion mas corta, que es lo que queriamos. Funciona con un limite basado en la profundidad actual (g) y lo que falta (h).

Diapo 7: Que es la heuristica (h)

Pongamonis en situacion, si IDA* no supiera a donde va, tardaria años. La heuristica es una estimacion de cuantos movimientos faltan como minimo. Creamos dos tablas gigantes, una para la permutacion y otra para la orientacion. Como son problemas simplificados, se calculan al principio y luego se consultan rapido.

Diapo 8: Por que usamos MAX y no sumamos

Aqui me lie un poco al principio. Tenemos la distancia de la permutacion y la de la orientacion. En vez de sumarlas, cogemos el valor maximo. Por que? Porque un mismo movimiento puede arreglar las dos cosas a la vez. Si sumaramos, nos pasariamos del coste real y la heuristica ya no seria segura, rompiendo el IDA*.

Diapo 9: Validar antes de volvernos locos

Antes de buscar una solucion, el programa hace validaciones. Si le metemos colores imposibles o piezas repetidas, nos da error de entrada en vez de tirarse horas buscando a lo tonto. Asi aseguramos que el cubo de verdad puede ser resuelto.

Diapo 10: Podas seguras

Para no hacer trabajo de mas, el codigo mete podas. Por ejemplo, si haces un giro R, no pruebas otro giro R justo despues porque seria tonteria. O giros opuestos que no afectan a lo mismo. Esto recorta el arbol de busqueda un monton.

Diapo 11: Ejecucion y pruebas

Para probarlo, podeis ejecutar el main.py. Tiene un menu donde podeis mezclar el cubo, ver la solucion, copiarla a un cubo fisico y ver como se resuelve de verdad. Es bastante facil e intuitiva de usar.

Diapo 12: Conclusiones y limites

En resumen, modelo el estado, busco con IDA* mas las tablas heuristicas y listo. Lo peor es meter a mano las 24 pegatinas si quereis resolver un cubo vuestro, pero con paciencia sale. Como he mencionado, GPT6 hizo el trabajo duro de teclear el codigo, pero entender como se juntan las piezas ha sido mi trabajo y espero que os haya resultado interesante. Gracias por la atención.

.